```python
ALGO_MAP = {
    'FCFS': {'csv': '../fcfs/FCFS.csv', 'dir': 'FCFS'},
    'PR': {'csv': '../priority/Priority.csv', 'dir': 'PR'},
    'RR': {'csv': '../round_robin_c/RR.csv', 'dir': 'RR'},
    'SJF': {'csv': '../sjf/SJF.csv', 'dir': 'SJF'},
    'SRTF': {'csv': '../srtf/SRTF.csv', 'dir': 'SRTF'}
}

def main():  rNADOMROOM22stopmotion
    print("Welcome to OS Scheduling Visualizer!")
    print("Available algorithms: FCFS, PR (Priority), RR (Round Robin), SJF, SRTF")

    try:
        algo_input = input("Please select an algorithm: ").strip().upper()
    except EOFError:
        print("\nExiting...")
        sys.exit(0)
    except KeyboardInterrupt:
        print("\nExiting...")
        sys.exit(0)

    if algo_input not in ALGO_MAP:
        print(f"Error: Unknown algorithm '{algo_input}'. Please choose from: FCFS, PR, RR, SJF, SRTF.")
        sys.exit(1)

    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_file_path = os.path.join(script_dir, ALGO_MAP[algo_input]['csv'])
    out_dir = os.path.join(script_dir, ALGO_MAP[algo_input]['dir'])

    if not os.path.exists(csv_file_path):
        print(f"Error: Could not find '{csv_file_path}'.")
        print("Please ensure you have run the corresponding C program first.")
        sys.exit(1)

    try:
        df = pd.read_csv(csv_file_path)
    except Exception as e:
        print(f"Error reading CSV file: {e}")
        sys.exit(1)

    if df.empty:
        print("The CSV file is empty. No data to visualize.")
        sys.exit(1)

    print(f"Data successfully loaded from {csv_file_path}. Found {len(df)} processes.")

    # Create output directory
    os.makedirs(out_dir, exist_ok=True)

    # Chart 1: Turnaround Time & Waiting Time per Process
    plt.figure(figsize=(10, 6))
    width = 0.35
    x = range(len(df))

    plt.bar([i - width/2 for i in x], df['Turnaround_Time'], width, label='Turnaround Time', color='skyblue')
    plt.bar([i + width/2 for i in x], df['Waiting_Time'], width, label='Waiting Time', color='salmon')

    plt.xlabel('Process ID (PID)')
    plt.ylabel('Time Units')
    plt.title(f'{algo_input} - Turnaround Time and Waiting Time per Process')
    plt.xticks(x, [f"P{pid}" for pid in df['PID']])
    plt.legend()
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    tat_wt_path = os.path.join(out_dir, 'tat_wt_chart.png')
    plt.savefig(tat_wt_path)
    print(f"Saved '{tat_wt_path}'")
    plt.close()
```

```python
    # Chart 2: Burst Time vs Turnaround Time
    plt.figure(figsize=(10, 6))
    plt.scatter(df['Burst_Time'], df['Turnaround_Time'], color='purple', s=100, alpha=0.7)

    # Add labels to points
    for i, row in df.iterrows():
        plt.annotate( text: f"P{row['PID']}",
                      xy: (row['Burst_Time'], row['Turnaround_Time']),
                      textcoords="offset points",
                      xytext=(0,10),
                      ha='center')

    plt.xlabel('Burst Time')
    plt.ylabel('Turnaround Time')
    plt.title(f'{algo_input} - Relationship: Burst Time vs Turnaround Time')
    plt.grid( visible: True, linestyle='--', alpha=0.7)

    scatter_path = os.path.join(out_dir, 'burst_vs_tat_scatter.png')
    plt.savefig(scatter_path)
    print(f"Saved '{scatter_path}'")
    plt.close()

    # Chart 3: Process Timeline (Arrival to Completion)
    plt.figure(figsize=(10, 6))

    for i, row in df.iterrows():
        plt.plot( *args: [row['Arrival_Time'], row['Completion_Time']], [i, i],
                  marker='o', linewidth=3, markersize=8, label=f"P{row['PID']}")

    plt.yticks(range(len(df)), [f"P{pid}" for pid in df['PID']])
    plt.xlabel('System Time')
    plt.title(f'{algo_input} - Process Execution Lifespan (Arrival → Completion)')
    plt.grid( visible: True, linestyle='--', alpha=0.4)
```

```python
    lifespan_path = os.path.join(out_dir, 'process_lifespan.png')
    plt.savefig(lifespan_path)
    print(f"Saved '{lifespan_path}'")
    plt.close()

    print("\nAll visualizations have been successfully generated!")

if __name__ == "__main__":
    main()
```

Output

```
(OS project) ┌─(...s/OS/OS project/algorithms/viz)──(dhruvrathod㉿Dhruvs-MacBook-Air:s000)─┐
└─(13:13:55 on main ✖ ● ★)──> uv run visualize_algo.py
Welcome to OS Scheduling Visualizer!
Available algorithms: FCFS, PR (Priority), RR (Round Robin), SJF, SRTF
Please select an algorithm: SRTF
Data successfully loaded from /Users/dhruvrathod/Documents/notes/OS/OS project/algorithms/viz/../srtf/SRTF.csv. Found 5 processes.
```

```
burst_vs_tat_scatter.png
process_lifespan.png
tat_wt_chart.png
```